LEXICON

APIs, part 1

Consuming APIs with Python

```
import requests

r = requests.get(
    "https://api.github.com"
)
print(r.status_code)  # 200
```

Python & AI · Developer programme 2026/2027

Today

LEXICON

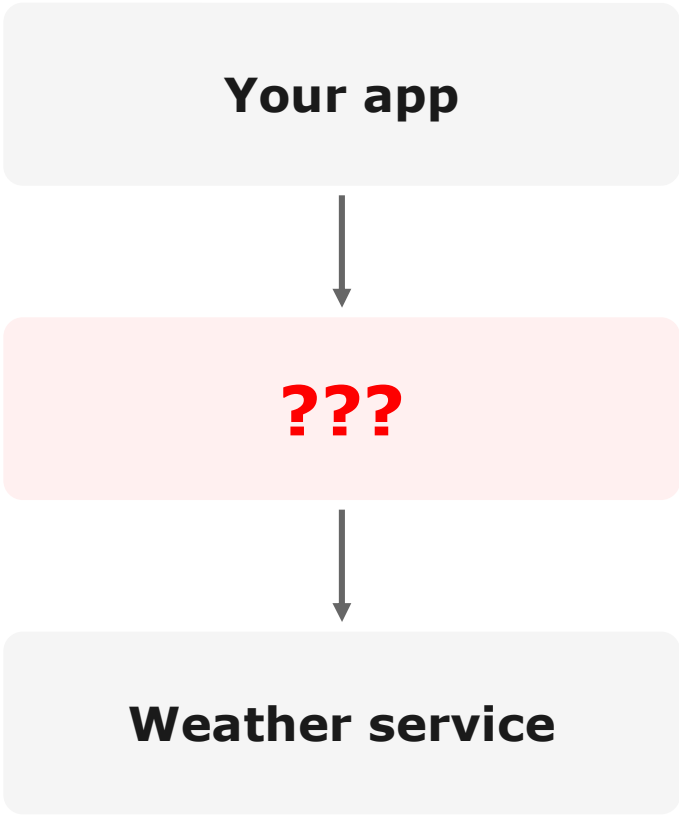
After today you can:

- Make a request to an API from Python
- Read the answer: JSON and status codes
- Handle errors without crashing
- Keep API keys and tokens safe

Warm-up question

Your weather app says 8°C in Stockholm.

Where does that number come from?



What is an API?



A set of rules

Lets programs talk to each other

One API, many clients

Website, mobile app, smartwatch

A safe middle layer

Nobody talks to the database directly

REST and JSON

LEXICON

REST
A style for APIs built on HTTP

Stateless
Every request carries everything the server needs

```
{
  "id": 1,
  "title": "Backpack",
  "price": 109.95
}
```

JSON	Python
object	**dict**
array	**list**
string	**str**
number	**int / float**
true / false	**True / False**
null	**None**

Anatomy of a request

LEXICON

Endpoint

Where: which resource

https://fakestoreapi.com/products

Method

What: read, create, change, delete

GET

Headers

Extra information about the request

Accept: application/json

Body

The data we send

{"title": "New product"}

Endpoint: root, path, query

LEXICON

```
https://api.github.com/users/octocat/repos?sort=pushed&per_page=5
```

ROOT	PATH	QUERY
Where the API lives	Which resource	Options: ? starts, & separates

Docs write placeholders like :username or {username}. Replace them with a real value.

Never put keys or passwords in the URL.

Methods and CRUD

LEXICON

Method	CRUD	What it does
GET	Read	Get data, change nothing
POST	Create	Add something new
PUT	Update	Replace the whole thing
PATCH	Update	Change only some fields
DELETE	Delete	Remove it

How an API uses the methods is up to the developer. Read the docs.

Status codes

LEXICON

2xx Success

3xx Redirect

4xx Your mistake

5xx Server problem

Code	Meaning
200	OK
201	Created
401	Unauthorized
403	Forbidden
404	Not found
422	Invalid data
429	Too many requests
500	Server error

Our first GET

LEXICON

LIVE 1

1. GET all products
2. Status, headers and .json()
3. Loop over the data
4. Path: /products/15
5. Query: params={"limit": 5}

```
response = requests.get(
    url, params={"limit": 5}
)
data = response.json()
```

Expected 200 and a list of 5 products

You may see

TypeError: 'Response' object is not subscriptable

Use response.json() first

When things go wrong

LEXICON

LIVE 2

What happens	What you get	What you catch
Server says no (404, 500)	**HTTPError**	raise_for_status()
Server too slow	**Timeout**	timeout=5
Cannot connect	**ConnectionError**	except ConnectionError
Answer is not JSON	**JSONDecodeError**	except JSONDecodeError
Anything else	**RequestException**	Catch it last

requests has no default timeout. Always set one.

Write to an API

LEXICON

LIVE 3

```
# Create
requests.post(url, json=new_product)
# Replace the whole product
requests.put(f"{url}/7", json=whole)
# Change only some fields
requests.patch(f"{url}/7", json=changes)
# Remove
requests.delete(f"{url}/6")
```

Expected Status 200 and the new id

json=

Sends JSON and sets Content-Type for you

data=

Sends form data. Wrong for JSON APIs

Fake API

Nothing is really saved. It only pretends

Who are you?

LEXICON

LIVE 4

Why identify yourself?

- Limit how much you use the API
- Charge for usage
- Keep data private

```
headers = {
   "Authorization": f"Bearer {token}"
}
requests.get(url, headers=headers)
```

401 **Unauthorized**
I do not know who you are

403 **Forbidden**
I know who you are, but no

Keep secrets secret

LEXICON

LIVE 4

Never in your code

Never in the URL

Never in Git

```
# .env
GITHUB_TOKEN=ghp_xxxxxxxx

# .gitignore
.env
```

```
load_dotenv()
token = os.getenv("GITHUB_TOKEN")
```

Pushed a key by mistake? Revoke it and make a new one.

Good to know

LEXICON

429

Too many requests. Look for a Retry-After header.

Retry-After: 30

Pagination

Big results come one page at a time.

?limit=5&page=2

Docs first

Endpoints, parameters and auth are in the docs.

/users/:username

Versions

APIs change, so the version is in the URL.

/v1/users

Remember

LEXICON

- ✓ A request has an endpoint, a method, headers and a body
- ✓ Always check the status code
- ✓ Set a timeout on every request
- ✓ json= for JSON, params= for query parameters
- ✓ Keys and tokens live in .env, never in Git

Tomorrow: your own API

LEXICON

1 Build an API with FastAPI

2 Validate data and return the right status codes

3 Protect it with an API key

```
@app.get("/products")
def list_products():
    return products
```

Before tomorrow: finish Lab 1 and Lab 2